实验 1 Linux 下的 C/C++编程环境

实验目的

1、掌握 vim 编辑器的使用
2、掌握编译软件 gcc 的使用
3、掌握 makefile 编写
4、掌握调试软件 gdb 使用

实验内容

1、输入、编辑、编译、执行 c 语言源程序。
2、使用 makefile 完成多文件项目编译
3、使用 gdb 对程序的执行进行跟踪、调试

实验指导

任务一：配置 vim 编辑器的工作环境

1、简介

vim 是 Linux 中常用的一个编辑器，使用者可以根据自己的需要对其工作环境进行配置。vim 工作环境的配置方法有如下几种：

(1) 在末行模式下使用命令“:set 配置项”、“:set 配置项=值”或者“配置项”等方式来进行配置。这种配置的方式的特点是一旦重新启动 vim 软件，原先所配置的信息将全部丢失。

(2) 修改 vim 配置文件“/etc/vim/vimrc”的内容，把各种配置命令写入到该文件中。该配置文件中本身已有一些配置项，其中以双引号开头的配置项已被注释、不起作用，若要启用对应的配置项，只要把开头双引号删除掉即可。这种使用配置文件“/etc/vim/vimrc”来配置 vim 工作环境的方法，其特点是该文件配置的工作环境对所有的用户都起作用。

(3) 修改用户家目录下的配置文件“~/.vimrc”。由于该文件的文件名以句点开始，所以它是一个隐藏文件。使用“~/.vimrc”配置 vim 工作环境的方法与使用配置文件“/etc/vim/vimrc”配置 vim 工作环境的方法相同。本方法的特点是配置文件“~/.vimrc”只对相应的用户起作用。

2、实验步骤

(1) 打开配置文件“~/.vimrc”，阅读其中各配置项及其注释。

(2) 将文件“~/.vimrc”改名为：“~/.vimrc.bak”。

(3) 使用 vim 创建并编辑“~/.vimrc”，其内容如下：

```
set nocompatible
```

```
syntax enable
syntax on
filetype indent on
set tabstop=4
set shiftwidth=4
set autoindent
set cindent
set backspace=2
set cindent shiftwidth=4
set smartindent
set number
set showmatch
set matchtime=5
set background=dark
set mouse=a
set ruler
set hlsearch
set incsearch
set nowrapscan
set nocompatible
set cursorline
set history=1000
set ignorecase
inoremap ( ()<LEFT>
inoremap [ []<LEFT>
inoremap { {<CR>}<UP>
inoremap < <><LEFT>
inoremap " ""<LEFT>
inoremap ' '<LEFT>
set wrap
set laststatus=2
set cmdheight=1
set shm+=I
```

(4) 重新启动 vim 编辑器，将以下代码输入到文件 hello.c 中，观察代码的输入过程以及代码显示情况。

```
#include <stdio.h>
int add(int a,int b);
void main(void)
{
    int x,y;
    printf("Please input two intcgcr:");
    scanf("%d %d",&x,&y);
    printf("%d plus %d answer is:%d\n",x,y,add(x,y));
```

```
        return;
    }
    int add(int a,int b)
    {
        int ans;
        ans=a+b;
        return ans;
    }
```

(5) 请解释第(3)步输入到文件“~/.vimrc”中每一个配置项的含义。

(6) 删除刚才创建的配置文件“~/.vimrc”，把前面保存的配置文件“~/.vimrc.bak”改回名字“~/.vimrc”。

任务二：vim 多窗口编辑

1、简介

vim 提供多窗口编辑的功能，即：可以在建立多个窗口来编辑文档，从而简化了复合的编辑任务。vim 的多窗口并不是说在终端上启动多个 vim 实例，而是一个 vim 实例中创建了多个窗口。vim 既可以沿水平方向分割窗口，也可以沿垂直方向分割窗口，还可以沿水平方向和垂直方向混合分割窗口。vim 各窗口可以编辑不同的文件，也可以编辑同一个文件。

vim 启动多窗口编辑的方法：

(1) 启动 vim 时打开多个窗口

启动 vim 时，可以通过使用选项“-o”或者“-O”来创建多个窗口。其中“-o”要求沿水平方向分割窗口，“-O”要求沿垂直方向分割窗口。比如：

```
vim -o file1 file2...
vim -O file1 file2...
```

若在选项“-o”或者“-O”后面加上一个数字，则指出以预分配窗口方式打开文件，而数字表示创建窗口的数量 。比如：

```
vim -o5 file1 file2
vim -O5 file1 file2
```

(2) 运行时动态创建多个窗口

在 vim 已经启动后，通过使用末行模式命令“:split”或者“:vsplit”，可以动态创建窗口。这两个命令的用法如下：

命令一：

```
:[N]sp[lit] [file]
```

该命令的作用是：以水平分割的方式创建一个新窗口，新窗口的高度为 N 行。

若省略数字 N，则将当前窗口平均分割为两个窗口。

若省略文件名 file，则在新窗口显示原来窗口窗口中打开的文件，即在两个窗口中显示同一个文件的内容。不省略文件名 file，则在新窗口中编辑指定的文件；若指定的文件还未装载到内存，则该命令会自动装载指定文件；若指定的文件不存在，则本窗口将会使用给出的文件名来保存在该窗口中编辑的文件。

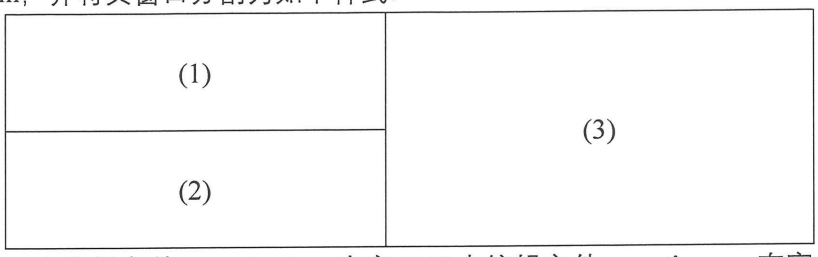
命令二：

```
:[N]vs[plit] [file]
```

该命令除了是要求沿垂直方向分割原窗口、N 指的是新窗口的宽度外，其它与命令“:split”相同。

2、实验步骤

(1) 启动 vim，并将其窗口分割为如下样式：



(2) 在窗口(1)中编辑文件 greeting.h，在窗口(2)中编辑文件 greeting.c，在窗口(3)中编辑文件 myapp.c。这三个文件的代码分别如下：

文件 greeting.h 的内容如下：

```
#ifndef _GREETING_H
#define _GREETING_H
    void greeting(char *name);
#endif
```

文件 greeting.c 的内容如下：

```
#include <stdio.h>
#include “greeting.h”
void greeting(char *name)
{
    printf(“Hello %s!\n\n”,name);
}
```

文件 myapp.c 的内容如下：

```
#include <stdio.h>
#include “greeting.h”
#define N 10
int main(void)
{
    char name[N];
    printf(“Enter your name,please:”);
    scanf(“%s”,name);
    greeting(name);
    return 0;
}
```

任务三：编译器 gcc 的使用

1、简介

GCC(GNU project C and C++ compiler)是 GNU 项目的一部分，最初是一个 C/C++语言的编译器，现在已经可以支持包括 C/C++、Objective-C、Pascal、Fortran、Ada 和 Java 等多种语言的开发，GCC 名称也随之改变为 GNU Compiler Collection。虽然如此，其最重要的目标还是 C/C++的开发。

GCC 可以支持交叉编译，即可以在 A 软硬件平台上编译生成运行在 B 软硬件平台的目标程序。GCC 支持的硬件平台有 X86、ARM、PowerPC、MIPS 等，支持的软件平台有：Linux、Windows、

Android、Solaris 等。

一个 C/C++程序从开始编码到生成可执行的二进制文件至少要经过 4 个步骤：

- 预处理(Preprocessing)
- 编译(Compilation)
- 汇编(Assembly)
- 链接(Linking)

通过给 gcc 设置不同的命令参数，可以使得程序生成到上述的任意一个步骤。所以使用不同的选项来启动 gcc 对程序编译，能够对每个细节进行精细控制，从而有利于代码优化、代码重用和错误定位。如果开发者不关心这些控制细节，也可以通过设置参数一次性地完成从源代码到二进制执行代码的生成，简化程序生成过程。

gcc 命令格式

gcc [选项] <文件名> ……

其中：

- 选项：对程序的生成进行控制
- 文件名：被处理的文件

经 gcc 处理生成的文件可能是预处理文件、汇编文件、目标文件或者最终的可执行文件。使用选项：-o<文件名>可指定保存经过 gcc 处理后结果文件的文件名。若省略了选项“-o<文件名>”，那么生成的各种文件默认的名字为：

- 汇编文件的主文件名与被处理文件的主文件名相同，扩展名分别为“.s”；
- 目标文件的主文件名与被处理文件的主文件名相同，扩展名分别为“.o”；
- 可执行文件的名称为“a.out”；
- 预处理文件发送到标准输出设备。

gcc 常用的选项有：

(1)选项：-E

只对源程序中如#include、#define 等预处理指令进行处理。不进行编译、汇编、链接等处理。如以下的命令将处理 C 语言源程序 hello.c 中的预处理指令，并把预处理后的结果保存到文件 hello.i 中。

```
gcc -E -o hello.i hello.c
```

(2)选项：-S

生成汇编语言文件后即停止，不要再继续生成目标文件。比如以下命令对 C 语言源程序 hello.c 编译，生成了汇编语言程序 hello.s。

```
gcc -S hello.c
```

(3)选项：-c

仅对源文件进行编译，不链接生成可执行文件。若只想对源程序查错，或只需产生目标文件，可以使用该选项。比如以下命令：

```
gcc –c file1.c file2.c file3.c
```

作用是分别对 file1.c、file2.c 和 file3.c 进行编译，生成二进制形式的目标文件。由于没有使用“-o <文件名>”选项，所以生成的目标文件名分别为 file1.o、file2.o 和 file3.o。

(4)选项：-g[gdb]

在可执行文件中加入调试信息，方便进行程序调试。如果使用中括号中的选项，表示要加入 gdb 扩展的调试信息，以方便使用 gdb 对程序进行调试。

(5)选项：-Idir

为#include 增加一个额外的搜索头文件的目录，该目录的名称为“dir”。比如：

```
gcc -I./mydir hello.c
```

(6)选项：-Ldir

在编译源程序时，增加一个搜索库文件的额外目录，该目录的名称为“dir”。

(7)选项：-llibrary

在编译链接程序时，增加一个额外的链接库“library.a”。

gcc 在编译的过程中，支持对多种类型的输入文件进行处理。输入文件的类型是通过输入文件的扩展名来进行识别的。根据输入文件的类型，也就确定了能对输入文件施加何种处理操作。gcc 常见文件类型的扩展名、类型及可以施加的处理方法如下表所示：

扩展名	文件类型	可施加的处理
.c	C 语言源程序	预处理、编译、汇编、链接
.C, .cc, .cp, .cpp, .C++, .cxx	C++语言源程序	预处理、编译、汇编、链接
.m	Objective-C 语言源程序	预处理、编译、汇编、链接
.i	预处理后的 C 语言源程序	编译、汇编、链接
.ii	预处理后的 C++语言源程序	编译、汇编、链接
.s	预处理后的汇编语言源程序	汇编、链接
.S	未经预处理的汇编语言源程序	预处理、汇编、链接
.o	目标文件	链接
.h	头文件	不进行任何操作
.a	静态库文件	不进行任何操作
.so	共享库文件	不进行任何操作

2、实验步骤

(1) 对任务二第(2)步输入的 c 语言源程序 myapp.c 进行如下操作：

- 对其做预处理操作，输出的预处理文件的名字为“myapp.i”；
- 对其进行预处理和编译操作，生成的汇编程序的名字为“myapp.s”

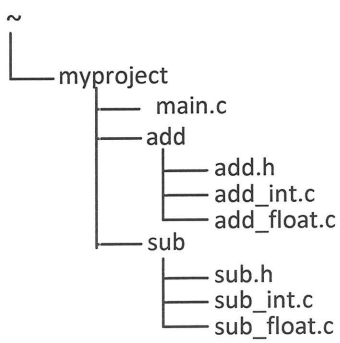
(2) 分别对任务二第(2)步输入的源程序 myapp.c 和 greeting.c 进行预处理、编译和汇编，生成二进制目标代码文件：myapp.o 和 greeting.o。

(3) 把上面第(2)步生成的目标代码文件链接成可以执行的程序 myapp。

(4) 在当前目录下创建一个目录“~/myproject”。然后再以该目录为当前目录，其下分别创建 7 个文件：

- main.c
- add/add.h
- add/add_int.c
- add/add_float.c
- sub/sub.h
- sub/sub_int.c
- sub/sub_float.c

其目录结构为：



这 7 个文件的内容分别为：

文件 main.c

```
/*main.c*/
#include <stdio.h>
#include “add.h”
#include “sub.h”
#define N 10
int main(void)
{
    int a = 10,b = 12;
    float x = 1.234567, y = 9.87654321;

    printf(“int a+b is:%d\n”,add_int(a,b));
    printf(“int a-b is:%d\n”,sub_int(a,b));
    printf(“float x+y is:%f\n”,add_float(x,y));
    printf(“float x-y is:%f\n”,sub_float(x,y));
    return 0;
}
```

文件 add.h

```
/*add.h*/
#ifndef __ADD_H__
#define __ADD_H__
extern int add_int(int a,int b);
extern float add_float(float x,float y);
#endif
```

文件 add_float.c

```
/*add_float.c*/
float add_float(float x,float y)
{
    return x+y;
}
```

文件 add_int.c

```
/*add_int.c*/
float add_float(int a,int b)
{
    return a+b;
}
```

文件 sub.h

```
/*sub.h*/
#ifndef __SUB_H__
#define __SUB_H__
extern int sub_int(int a,int b);
extern float sub_float(float x,float y);
```

```
#endif
```

文件 sub_float.c

```
/*sub_float.c*/
float sub_float(float x,float y)
{
    return x-y;
}
```

文件 sub_int.c

```
/*sub_int.c*/
float sub_float(int a,int b)
{
    return a-b;
}
```

(5) 使用命令行方式分别对第(4)步输入的 5 个 c 语言源程序编译生成目标代码文件，然后再将这 5 个目标文件链接为可执行的二进制程序 cacu1。

(6) 使用一条命令，将上述 C 语言源程序直接生成可执行的二进制程序 cacu2。

任务四：make 命令的使用

1、简介

将一个大的项目分解为多个小的部分，分别对每一个部分进行编码和调试，这样可以提高软件调试和维护的效率。但是，这种组织方式也使得对程序的编译变得复杂，使用的编译命令增多。每次对程序源代码进行改进之后，都要执行多条命令来重新编译程序，这对程序开发人员无疑是很大的负担。为了简化生成代码的步骤，GNU 提供了 make 工具。

make工具读入一个位于当前目录下、名称为“makefile”文本文件，通过执行该文件中的内容来完成相关的处理工作。被Make命令读入文件的内容由一系列的规则构成，这些规则说明了生成最终的二进制代码依赖哪些模块，以及这些模块如何生成。Make命令执行时，先在makefile文件中查找各种规则，对规则进行解析后再运行规则。规则的一般格式是：

```
<目标>:<依赖项列表>
<制表符>命令
…
```

其中：

- <目标>：规则所定义的目标。通常规则是最后生成的可执行文件的文件名或者为了生成可执行文件而依赖的目标文件的文件名。
- <依赖项列表>：执行本规则所必需的依赖条件项列表，各条件之间使用空格隔开。一般为生成目标所需的其他规则的目标名称或者文件名。
- <命令>：规则所执行的命令，即规则的动作，比如编译文件、生成库文件、进入目录等。可以有多条命令，每条命令独占一行，每一行命令均要以制表符(Tab)开头。

make工具根据读入的文本文件来判断每个目标中的依赖项是否是最新的，若不是最新的就根据其后的命令生成最新的依赖项，从而保证目标为最新的。与手工编译和链接相比，make命

令的优点在于只更新修改过的文件。在Linux中，创建或更新文件后，系统会记录最后的修改时间，make命令就是通过这个最后修改时间来判断此文件是否被修改过，因此不会漏掉任何需要更新的文件。

比如对于任务二的实验步骤(2)中输入的 greeting.h、greeting.c、myapp.c 可以通过编写如下的makefile、使用 make 命令来进行编译，其中 greeting.h 和 greeting.c 保存在当前目录下的 fun 目录中。

```
myapp:greeting.o myapp.o
    gcc –o myapp greeting.o myapp.o
greeting.o:fun/greeting.h fun/greeting.c
    gcc –c fun/greeting.c
myapp.o:myapp.c fun/greeting.h
    gcc –c myapp.c –Ifun
clean:
    rm –f *.o
    rm –f myapp
```

说明:

- 第一条规则：该规则的名字为myapp，该规则依赖于另外两条规则：greeting.o和myapp.o。make命令首先会检查是否存在greeting.o，若不存在greeting.o文件，则先转去执行规则greeting.o，以生成greeting.o文件；然后再检查是否存在myapp.o文件，若不存在myapp.o文件，则转去执行myapp.o规则以生成myapp.o文件；当在当前目录下已生成greeting.o和myapp.o这两个文件后，再执行命令："gcc –o myapp greeting.o myapp.o"，生成可执行文件myapp。
- 第二条规则：该规则的名字为greeting.o，该规则检查否存在文件"fun/greeting.h"和"fun/greeting.c"，若存在这两个文件，则执行命令："gcc –c fun/greeting.c"，生成目标文件greeting.o。
- 第三条规则：规则名字为myapp.o，该规则检查否存在文件myapp.c和fun/greeting.c，若存在这两个文件，则执行命令："gcc –c myapp.c –Ifun"，生成目标文件myapp.o。
- 第四条规则：规则名字为clean，无依赖项。该规则用于清理工程，删除当前目录下所有扩展为".o"的文件和文件"myapp"。

编写好makefile文件后，在makefile文件所在目录下，使用命令行方式执行以下命令，即可按照makefile文件中的规则进行动作。

```
make [规则名称]
```

若省略规则名称，则执行makefile中的第一条规则，否则，执行由规则名称所指定的规则。比如对于上述的makefile文件，若使用以下命令:

```
make
```

则执行"myapp"规则，对源程序进行编译，若使用以下命令:

```
make clean
```

则执行"clean"规则进行清理，执行该规则会删除当前目录下所扩展名为".o"的所有文件和myapp 文件。

2、实验步骤

(1) 编写一个 makefile 文件，完成对任务三实验步骤(4)中的 C 程序进行编译，并生成可以执

行的二进制目标代码程序。